

Line-by-Line Mathematical Guide for HAETAE_FIPS202.ec

Generated HAETAE EasyCrypt proof guide

2026-05-11

File Role

Modeled FIPS202/SHAKE support.

How To Read This Script

Read this file as deterministic hash-interface support for the model, not as a standard-model security proof for SHAKE.

Line-by-Line Mathematical Reading

Each row preserves one EasyCrypt source line and gives the intended mathematical or proof-script reading. Blank rows are kept because they mark proof structure in long EasyCrypt developments.

Line	EasyCrypt source	Mathematical reading
1	<code>require import AllCore IntDiv List.</code>	Imports AllCore IntDiv List. Mathematically, this exposes earlier carrier sets, operations, games, and lemmas that the current file may cite.
2	<code>require import HAETAE_Keccak1600.</code>	Imports HAETAE_Keccak1600. Mathematically, this exposes earlier carrier sets, operations, games, and lemmas that the current file may cite.
3	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
4	<code>theory HAETAE_FIPS202.</code>	Begins the namespace HAETAE_FIPS202, grouping the declarations as one checked theory.
5	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
6	<code>import HAETAE_Keccak1600.</code>	Opens HAETAE_Keccak1600 so its names can be used without qualification in the following proof.
7	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
8	<code>type fips_byte = int.</code>	Declares carrier set fips_byte. EasyCrypt will type-check all later functions and games over this mathematical domain.
9	<code>type fips202_state = fips_byte list.</code>	Declares carrier set fips202_state. EasyCrypt will type-check all later functions and games over this mathematical domain.
10	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
11	<code>op fips202_state_bytes : int = 200.</code>	Defines integer-valued parameter fips202_state_bytes, used for dimensions, budgets, support sizes, or bounds.
12	<code>op shake256_rate_bytes : int = 136.</code>	Defines integer-valued parameter shake256_rate_bytes, used for dimensions, budgets, support sizes, or bounds.
13	<code>op shake256_capacity_bytes : int = 64.</code>	Defines integer-valued parameter shake256_capacity_bytes, used for dimensions, budgets, support sizes, or bounds.
14	<code>op shake256_domain_separator : fips_byte = 31.</code>	Defines operation shake256_domain_separator, a total mathematical function or abbreviation used by later declarations.

Line	EasyCrypt source	Mathematical reading
15	op fips202_final_padding_byte : fips_byte = 128.	Defines operation fips202_final_padding_byte, a total mathematical function or abbreviation used by later declarations.
16	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
17	op fips202_keccak_f1600 (st : fips202_state) : fips202_state =	Defines operation fips202_keccak_f1600, a total mathematical function or abbreviation used by later declarations.
18	keccak_f1600_bytes st.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
19	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
20	op shake256_absorb_once_short_domain	Defines operation shake256_absorb_once_short_domain, a total mathematical function or abbreviation used by later declarations.
21	(_ : fips_byte list) (inlen : int) : bool =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
22	0 <= inlen /\ inlen < shake256_rate_bytes.	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
23	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
24	op shake256_absorb_once_short_block_byte	Defines operation shake256_absorb_once_short_block_byte, a total mathematical function or abbreviation used by later declarations.
25	(input : fips_byte list) (inlen i : int) : fips_byte =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
26	if 0 <= i /\ i < inlen then nth 0 input i	Splits proof obligations according to a conditional branch in the program or formula.
27	else if i = inlen /\ i = shake256_rate_bytes - 1 then	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
28	shake256_domain_separator + fips202_final_padding_byte	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
29	else if i = inlen then shake256_domain_separator	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
30	else if i = shake256_rate_bytes - 1 then fips202_final_padding_byte	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
31	else 0.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
32	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
33	op shake256_absorb_once_short_state	Defines operation shake256_absorb_once_short_state, a total mathematical function or abbreviation used by later declarations.
34	(input : fips_byte list) (inlen : int) : fips202_state =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
35	mkseq	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
36	(fun i =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
37	if i < shake256_rate_bytes	Splits proof obligations according to a conditional branch in the program or formula.
38	then shake256_absorb_once_short_block_byte input inlen i	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
39	else 0)	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
40	fips202_state_bytes.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
41	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
42	op shake256_absorb_once	Defines operation shake256_absorb_once, a total mathematical function or abbreviation used by later declarations.
43	(input : fips_byte list) (inlen : int) : fips202_state =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
44	fips202_keccak_f1600	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
45	(shake256_absorb_once_short_state input inlen).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
46	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
47	op shake256_squeeze (st : fips202_state) (outlen : int) : fips_byte list =	Defines integer-valued parameter shake256_squeeze, used for dimensions, budgets, support sizes, or bounds.

Line	EasyCrypt source	Mathematical reading
48	<code>mkseq (fun i => nth 0 st i) outlen.</code>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
49	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
50	<code>op shake256</code>	Defines operation shake256, a total mathematical function or abbreviation used by later declarations.
51	<code>(input : fips_byte list) (inlen outlen : int) : fips_byte list =</code>	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
52	<code>shake256_squeeze (shake256_absorb_once input inlen) outlen.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
53	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
54	<code>lemma fips202_state_bytesE :</code>	States checked lemma fips202_state_bytesE. The following proof script must derive this proposition from prior definitions and lemmas.
55	<code>fips202_state_bytes = 200.</code>	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
56	<code>proof. by rewrite /fips202_state_bytes. qed.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
57	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
58	<code>lemma shake256_rate_bytesE :</code>	States checked lemma shake256_rate_bytesE. The following proof script must derive this proposition from prior definitions and lemmas.
59	<code>shake256_rate_bytes = 136.</code>	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
60	<code>proof. by rewrite /shake256_rate_bytes. qed.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
61	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
62	<code>lemma shake256_capacity_bytesE :</code>	States checked lemma shake256_capacity_bytesE. The following proof script must derive this proposition from prior definitions and lemmas.
63	<code>shake256_capacity_bytes = 64.</code>	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
64	<code>proof. by rewrite /shake256_capacity_bytes. qed.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
65	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
66	<code>lemma shake256_domain_separatorE :</code>	States checked lemma shake256_domain_separatorE. The following proof script must derive this proposition from prior definitions and lemmas.
67	<code>shake256_domain_separator = 31.</code>	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
68	<code>proof. by rewrite /shake256_domain_separator. qed.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
69	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
70	<code>lemma fips202_final_padding_byteE :</code>	States checked lemma fips202_final_padding_byteE. The following proof script must derive this proposition from prior definitions and lemmas.
71	<code>fips202_final_padding_byte = 128.</code>	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
72	<code>proof. by rewrite /fips202_final_padding_byte. qed.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
73	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
74	<code>lemma shake256_rate_capacity_state_bytesE :</code>	States checked lemma shake256_rate_capacity_state_bytesE. The following proof script must derive this proposition from prior definitions and lemmas.
75	<code>shake256_rate_bytes + shake256_capacity_bytes = fips202_state_bytes.</code>	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
76	<code>proof.</code>	Begins the proof script for the preceding proposition.
77	<code>by rewrite /shake256_rate_bytes /shake256_capacity_bytes</code>	Discharges the current goal in one step using the cited simplification, lemma, or automation.
78	<code>/fips202_state_bytes.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
79	<code>qed.</code>	Ends the proof; EasyCrypt has accepted all remaining obligations.
80	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
81	<code>lemma shake256_absorb_once_short_state_size input inlen :</code>	States checked lemma shake256_absorb_once_short_state_size. The following proof script must derive this proposition from prior definitions and lemmas.

Line	EasyCrypt source	Mathematical reading
82	<code>size (shake256_absorb_once_short_state input inlen) =</code>	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
83	<code>fips202_state_bytes.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
84	<code>proof.</code>	Begins the proof script for the preceding proposition.
85	<code>by rewrite /shake256_absorb_once_short_state size_mkseq.</code>	Discharges the current goal in one step using the cited simplification, lemma, or automation.
86	<code>qed.</code>	Ends the proof; EasyCrypt has accepted all remaining obligations.
87	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
88	<code>lemma shake256_squeeze_size st outlen :</code>	States checked lemma <code>shake256_squeeze_size</code> . The following proof script must derive this proposition from prior definitions and lemmas.
89	<code>0 <= outlen =></code>	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
90	<code>size (shake256_squeeze st outlen) = outlen.</code>	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
91	<code>proof.</code>	Begins the proof script for the preceding proposition.
92	<code>move=> ge0_outlen.</code>	Introduces universally quantified variables or hypotheses into the proof context.
93	<code>rewrite /shake256_squeeze size_mkseq.</code>	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
94	<code>by smt().</code>	Discharges the current goal in one step using the cited simplification, lemma, or automation.
95	<code>qed.</code>	Ends the proof; EasyCrypt has accepted all remaining obligations.
96	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
97	<code>lemma shake256_size input inlen outlen :</code>	States checked lemma <code>shake256_size</code> . The following proof script must derive this proposition from prior definitions and lemmas.
98	<code>0 <= outlen =></code>	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
99	<code>size (shake256 input inlen outlen) = outlen.</code>	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
100	<code>proof.</code>	Begins the proof script for the preceding proposition.
101	<code>move=> ge0_outlen.</code>	Introduces universally quantified variables or hypotheses into the proof context.
102	<code>by rewrite /shake256 shake256_squeeze_size.</code>	Discharges the current goal in one step using the cited simplification, lemma, or automation.
103	<code>qed.</code>	Ends the proof; EasyCrypt has accepted all remaining obligations.
104	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
105	<code>end HAETAE_FIPS202.</code>	Closes the current theory or module block.